S2D-02: Locating Logs in Dynatrace

> **Series:** S2D — Splunk to Dynatrace Migration | **Notebook:** 2 of 9 | **Created:** January 2026 | **Last Updated:** 11/11/2026

Overview

Before migrating Splunk objects such as dashboards, alerts, or reports, it is essential to verify that the log data referenced in these objects is available in Dynatrace.

This notebook describes the process of determining where the required logs can be found, then validating whether they are present in Dynatrace.

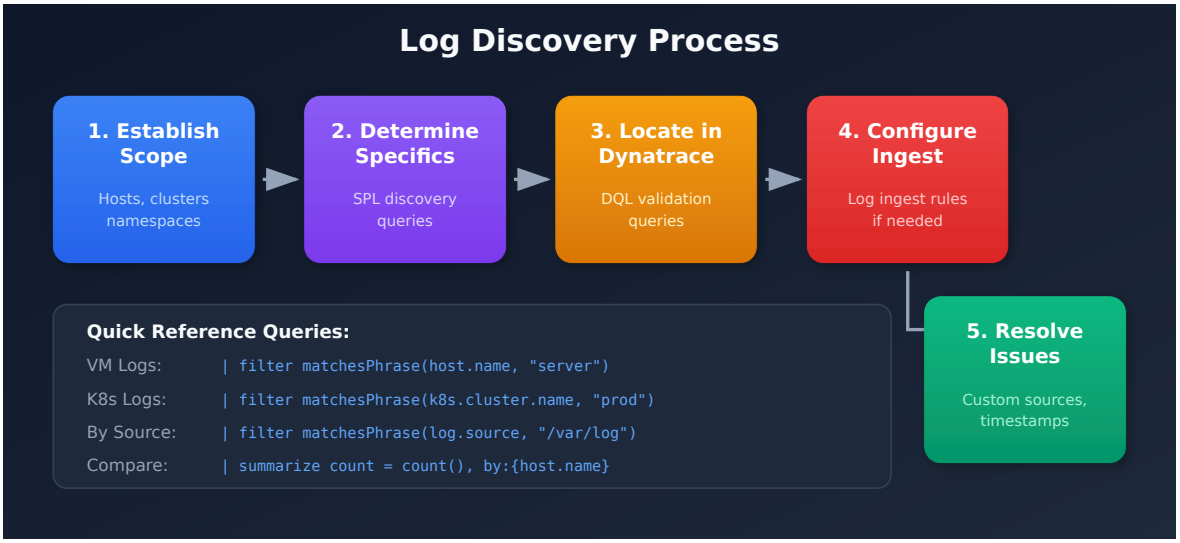


Table of Contents

1. Step 1: Establishing Scope
2. Step 2: Determining Specifics in Splunk
3. Step 3: Locating Logs in Dynatrace
4. Step 4: Configuring Log Ingest
5. Step 5: Resolving Discrepancies

6. Troubleshooting Checklist
7. Validation Query Template

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail
Permissions	`storage:logs:read`, `storage:buckets:read`, `settings:objects:read`
Splunk Access	Ability to run queries against source indexes
Infrastructure Knowledge	Understanding of application deployment topology

Learning Objectives

By the end of this notebook, you will be able to:

1. Establish the infrastructure scope for log migration
2. Use Splunk queries to identify specific log sources
3. Validate log availability in Dynatrace using DQL
4. Configure log ingest rules when data is missing
5. Troubleshoot common log ingestion issues

Step 1: Establishing Scope

When approaching a new application for migration, the first task is determining where the logs required for monitoring are located. This often coincides with the application's infrastructure, but not always.

Common Log Locations

Environment Type	Log Location Identifier
Virtual Machines	Host or host group
Kubernetes	Cluster, namespace, deployment
Cloud Functions	Cloud provider, function name
Containers	Container name, image

Gathering Scope Information

Information about application scope can come from:

1. **Management Zones** - If configured, these often define application boundaries
2. **Application Teams** - Direct knowledge of deployment topology
3. **CMDB/Inventory** - Infrastructure documentation
4. **Existing Splunk Queries** - Filters used in current dashboards

Step 2: Determining Specifics in Splunk

Not all logs within an application's footprint are required for every dashboard and alert. Conversely, dashboards and alerts may reference logs outside the expected footprint.

Splunk Discovery Queries

Run these queries in Splunk to understand the specific log sources being used:

Record Count by Host

```
index=[your_index] | stats count by host
```

Record Count by Source

```
index=[your_index] | stats count by source, sourcetype
```

Record Count by Kubernetes Attributes

```
index=[your_index] | stats count by kubernetes_cluster, kubernetes_namespace,
kubernetes_deployment_name
```

> **Tip:** If your Splunk environment doesn't use index-based organization, substitute the filters from your actual queries before the `stats` command.

Step 3: Locating Logs in Dynatrace

To determine whether required logs are present in Dynatrace, use DQL queries with the relevant filters from your Splunk discovery.

Search by Host Name

```
// Search logs by host name
// Replace 'your-host-name' with the actual host from Splunk
fetch logs, from:-1h
| filter matchesPhrase(host.name, "your-host-name")
| summarize count = count(), by:{log.source}
| sort count desc
```

Search by Log Source Path

```
// Search by specific log file path
// Replace path with your application's log location
fetch logs, from:-1h
| filter matchesPhrase(log.source, "/var/log/application")
| summarize count = count(), by:{host.name}
| sort count desc
```

Search by Kubernetes Attributes

```
// Search by Kubernetes cluster and namespace
fetch logs, from:-1h
| filter matchesPhrase(k8s.cluster.name, "production-cluster")
| filter matchesPhrase(k8s.namespace.name, "ecommerce")
| summarize count = count(), by:{k8s.deployment.name, k8s.pod.name}
| sort count desc
```

Compare Counts with Splunk

Run equivalent time-bounded queries in both platforms to compare log volumes.

> **Exact names for exact counts.** `matchesPhrase(host.name, "app-server-01")` also matches `my-app-server-01`, which Splunk `host=app-server-01` does not. It is fine for discovery; for a like-for-like count, switch to `host.name == "your-exact-host-name"` once you know the name.

```
// Total log count for the last hour (compare with Splunk)
fetch logs, from:now()-1h
| filter matchesPhrase(host.name, "your-host-name")
| summarize total_count = count()
```

Step 4: Configuring Log Ingest

If the required logs are not available in Dynatrace, you need to configure log ingest rules.

Ingest Rule Hierarchy

Level	Scope	Best For
Environment	All hosts	Organization-wide policies
Host Group	Specific host groups	Application-specific rules
Kubernetes cluster	One cluster	K8s-wide policies
Host	Single host	Special cases

> **Recommendation:** Use host group-level rules for application granularity without per-host overhead.

Log Ingest Rule Properties

The log ingest rules page states: *"Four hierarchy scopes are supported: host, Kubernetes cluster, host group, and environment."* (An earlier sentence on the same page still says three.)

Rules can filter logs based on:
- Host tag (key=value, set via `oneagentctl` or remote configuration). There is no host-name matcher.
- Log source (file name or path) and log source origin
- Kubernetes attributes (container, namespace, deployment, pod, workload)
- Log record level
- Process group
- Journald unit
- Log content (wildcards)

Documentation Reference

See Log Ingest Rules for complete configuration options.

Step 5: Resolving Discrepancies

If logs are still not appearing after configuring ingest rules, investigate these common issues:

Custom Log Sources

If your application writes to non-standard log locations, you may need to configure Custom Log Sources.

OneAgent Log Monitoring

Verify that log content access is enabled on the OneAgent. The `oneagentctl` command can check this:

```
oneagentctl --get-app-log-content-access
```

To enable it, run `oneagentctl --set-app-log-content-access=true`. The OneAgent CLI page says: *"Use the --get-app-log-content-access parameter to check whether Log Monitoring is enabled"* and *"Set the --set-app-log-content-access parameter to true or false"*.

> **Note:** Host-level settings supersede environment configuration. If log monitoring is disabled on the OneAgent, environment-level rules will not apply.

Timestamp and Splitting Issues

If logs appear but counts differ or content seems incorrect, you may need Timestamp and Splitting Rules to properly parse multi-line logs or custom timestamp formats.

Check what OneAgent discovered first

> **Coverage vs. volume — the log-module self-monitoring events.** Before assuming a missing Splunk log needs a custom source or a new ingest rule, check what OneAgent already discovered — the gap is often a source detected but not yet configured for ingest, not a discovery miss. The log module emits a `log_source.status` event per discovered source into `dt.system.events`, carrying `log.source.file_status` and `log.source.ingest_status` — so it reveals sources OneAgent **detected but is not ingesting**, which a `fetch logs` count cannot show (no stored records means nothing to count). It scans events, not raw logs, and needs only event-read.
>
> Log-module self-monitoring is enabled by default for Grail environments since SaaS 1.340 (*"Log module self-monitoring enabled by default for Grail"*), and it remains opt-in for Log Monitoring Classic. The built-in *Log ingest Overview* dashboard visualizes it. Verify the events are flowing in your tenant before relying on them. See **FAQ-08** (Recommended Approach) for the full file-status × ingest-status coverage matrix and the `fetch logs` fallback for tenants where the events are not available.
>
> **Quiet a warning you have already accepted, and only that one.** A migration surfaces SFM warnings for sources you have investigated and decided to leave as they are. Suppression rules are set through the Settings API with the `builtin:logmonitoring.log-sfm-settings` schema, at tenant, host-group,

> Kubernetes-cluster or host scope. Keep each rule narrow, for example one event type on one log source, so the same warning still fires elsewhere. Forthcoming in **SaaS 1.351** (pre-release, planned rollout from 11/11/2026): a UI for the same rules under **Settings > Collect and capture > Log monitoring > Configure log module > Advanced settings > SFM Events**. Until it reaches your tenant, the Settings API is the working path.
>
> **Sources:**
>
> - SaaS 1.340 (DT docs)
> - Monitor log source health with SFM events (DT docs) — *"To customize this functionality, update your configuration via the Settings API. Use the builtin:logmonitoring.log-sfm-settings schema ID."*; *"Keep the suppression scope as narrow as needed."*
> - SaaS 1.351 (DT docs) — *"Rollout start on Nov 11, 2026 (planned)"*; *"You can configure self-monitoring event suppression without working directly with the configuration API."*

```
// Log-source coverage from the OneAgent log-module self-monitoring events.
// Unlike `fetch logs` (which sees only sources that produced records), this
// surfaces sources OneAgent DETECTED but is NOT ingesting. Scans events, not
raw logs.
fetch dt.system.events, from: now() - 24h
| filter event.provider == "Log Module" and event.type == "log_source.status"
| dedup event.id, sort:{ timestamp desc }
| filter event.status == "Active"
| fieldsAdd coverage =
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Ingested", "Available | Ingesting", else:
    if(log.source.file_status == "FILE_STATUS_OK" and log.source.ingest_status
== "Not ingested", "Available | NOT ingesting", else:
    if(log.source.ingest_status == "Partially ingested", "Partial ingestion",
    else: "Unsupported or issue")))
| summarize sources = count(), by:{coverage}
| sort sources desc
```

Troubleshooting Checklist

Symptom	Possible Cause	Solution
No logs from host	Ingest rule missing	Add ingest rule for host/host group
No logs from file	Custom log source needed	Configure custom log source

Symptom	Possible Cause	Solution
Some logs missing	OneAgent disabled	Enable log content access
Wrong log count	Timestamp parsing	Configure timestamp rules
Multi-line broken	Splitting rules	Configure splitting rules

Validation Query Template

Use this template to create a comprehensive validation query for your application:

```
// Comprehensive log validation template
// Modify filters to match your application
fetch logs, from:now()-24h
| filter matchesPhrase(host.name, "app-server") or
matchesPhrase(k8s.namespace.name, "app-namespace")
| summarize {
    total_count = count(),
    error_count = countIf(status == "ERROR"),
    warn_count = countIf(loglevel == "WARN")
  }, by:{log.source}
| sort total_count desc
```

Next Steps

Once you've validated that required logs are available in Dynatrace, proceed to **S2D-03: SPL to DQL Translation** to begin converting your Splunk queries.

References

- Log Ingest Rules
- Custom Log Sources
- OneAgent CLI — log content access (DT docs)
- SaaS 1.340 (DT docs)
- Timestamp Configuration

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.